

DAY 4

1. Quoting

Definition

Quoting is the process of removing the **special meaning** of characters so Bash treats them as literal characters.

There are **3 primary types**:

① Backslash \

- Removes the special meaning of the **immediately following character**.
- Only the next character is affected.

Example:

```
echo \ $HOME
```

Output:

```
$HOME
```

Remember: \ → escape the next character

② Single Quotes ' '

- Remove the special meaning of **all characters inside** the quotes.
- Everything is treated literally.

Example:

```
echo '$HOME'
```

Output:

```
$HOME
```

`$HOME` is **not** expanded.

Remember: `' '` → everything literal

③ Double Quotes `" "`

- Remove the special meaning of **most characters**.
- However, `$` and backticks ``` retain their special meaning.
- A backslash can be used inside double quotes to escape these characters.

Example:

```
name="John"
echo "$name"
```

Output:

```
John
```

To print `$` literally:

```
echo "\$name"
```

Output:

```
$name
```

Quick Comparison

Quoting	What happens?
<code>\</code>	Escapes the next character
<code>'...'</code>	Everything is literal
<code>"..."</code>	Most characters are literal, but <code>\$</code> and <code>`</code> remain special

2. Tokenization

Definition

Tokenization is the **first step** Bash performs when processing a command.

It breaks a command line into smaller units called **tokens**.

What is a Token?

A **token** is a sequence of characters that Bash considers a **single unit**.

For example:

```
echo Hello World
```

The command contains separate tokens:

```
echo  
Hello  
World
```

3. Meta-characters

Bash uses certain **meta-characters** to identify and separate tokens.

Important meta-characters include:

```
| & ; ( ) < > space tab newline
```

Important concept

Unquoted meta-characters have their special meaning.

For example:

```
echo Hello; echo Bye
```

The `;` separates the two commands.

But:

```
echo "Hello; Bye"
```

Here `;` is inside quotes, so it is treated as **literal text**.

Quoting can prevent a meta-character from performing its special function.

4. Categorizing Tokens

After Bash identifies tokens using **unquoted meta-characters**, tokens are divided into two major categories:

A. Words

A token is a **word** when it does **not contain an unquoted meta-character**.

Example:

```
echo Hello
```

Here:

- `echo` → word
- `Hello` → word

B. Operators

A token is an **operator** when it contains at least one **unquoted meta-character**.

Examples:

```
|  
&&  
;  
||  
>  
<
```

```
>>  
<<
```

These operators have special functions in Bash.

5. Types of Operators

Bash operators can be broadly divided into **two functional types**.

① Control Operators

Control operators control the **execution flow** of commands.

Common examples:

Operator	Purpose
	Pipe output to another command
&&	Run next command if previous succeeds
	Run next command if previous fails
;	Separate commands
()	Group commands

Examples

Pipe:

```
ls | grep txt
```

The output of `ls` becomes the input of `grep`.

AND:

```
mkdir test && cd test
```

`cd test` runs only if `mkdir test` succeeds.

OR:

```
mkdir test || echo "Failed"
```

The `echo` command runs if `mkdir` fails.

6. Redirection Operators

Redirection operators control where **input** and **output** go.

Common operators:

Operator	Meaning
<	Take input from a file
>	Send output to a file, overwriting it
>>	Append output to a file
<<	Here-document / provide multiple lines of input

Examples

Output redirection:

```
echo "Hello" > file.txt
```

Writes `Hello` to `file.txt`.

Append:

```
echo "World" >> file.txt
```

Adds `World` to the end of the file.

Input:

```
sort < file.txt
```

Takes input from `file.txt`.
